

Estrutura de Dados

Inteligência Artificial e Ciência de Dados | 2026/1



Prof Vitor

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Introdução

- Você já parou para pensar em como medir a eficiência de um algoritmo?
- Muitas vezes pensamos em medir o tempo de um algoritmo. Mas isso significa que o **seu código** é eficiente?
- Para medir isso, usando a **complexidade de algoritmos**. ◀

Introdução

- Para calcular a complexidade de um algoritmo, usamos a **notação assintótica**.
- A notação assintótica é usada para **medir** como o **tempo de execução** ou o espaço de memória de seu algoritmo **cresce à medida que o tamanho da entrada cresce**.

Introdução

- Por **tamanho da entrada**, entedemos a quantidade de dados que seu algoritmo recebe. Se a entrada é um vetor de 10 elementos, o tamanho é 10 e assim por diante.
- O problema de medir o tempo (em segundos/milisegundos) é que isso é fortemente afetado pelo hardware do computador. Ou seja, não é sobre a eficiência do código.

Big O

- Queremos descobrir, então, a **ordem de grandeza da quantidade de passos executados** pelo algoritmo de acordo com o tamanho da entrada.
- Chamamos ordem de grandeza de **Big O**.



Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def soma_elementos(lista, n):  
    soma = 0  
    i = 0  
  
    while i < n:  
        soma += lista[i]  
        i += 1  
  
    return soma
```

1

1

n+1

n

n

1

podemos afirmar que

$$t(n) = 4 + 3n$$

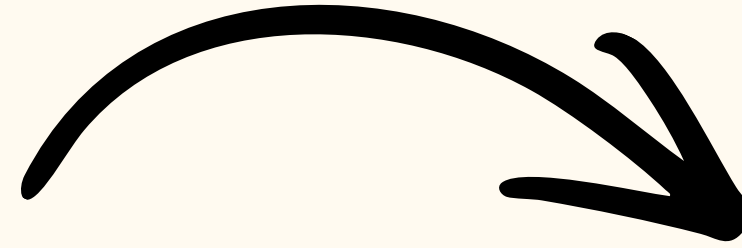
podemos afirmar que

$$t(n) = 4 + 3n$$

logo, se tivermos uma
entrada de 10 elementos

$$t(10) = 4 + 3 \times 10$$

$$t(10) = 34$$

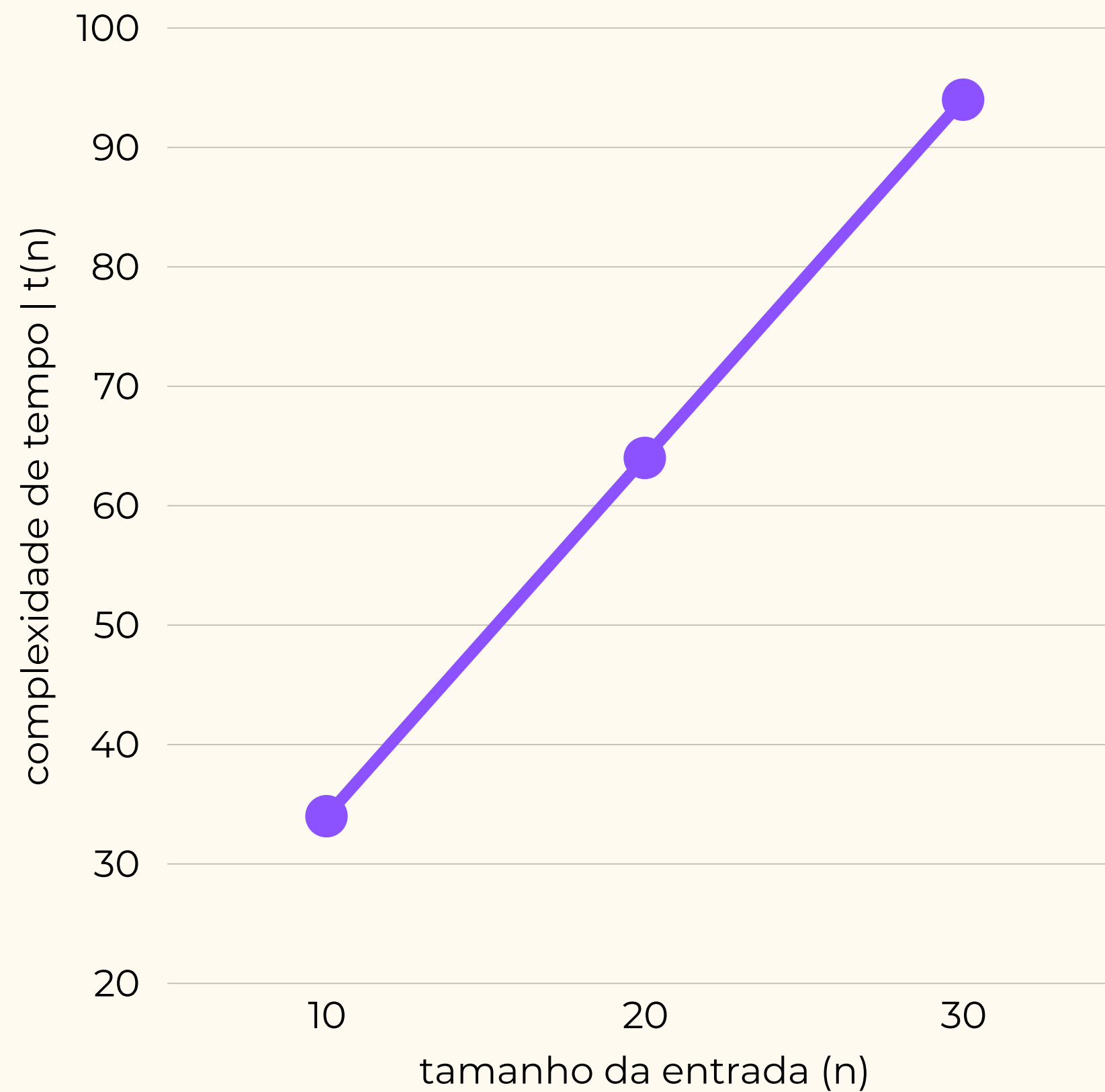


e assim por diante...

n	t(n)
10	34
20	64
30	94

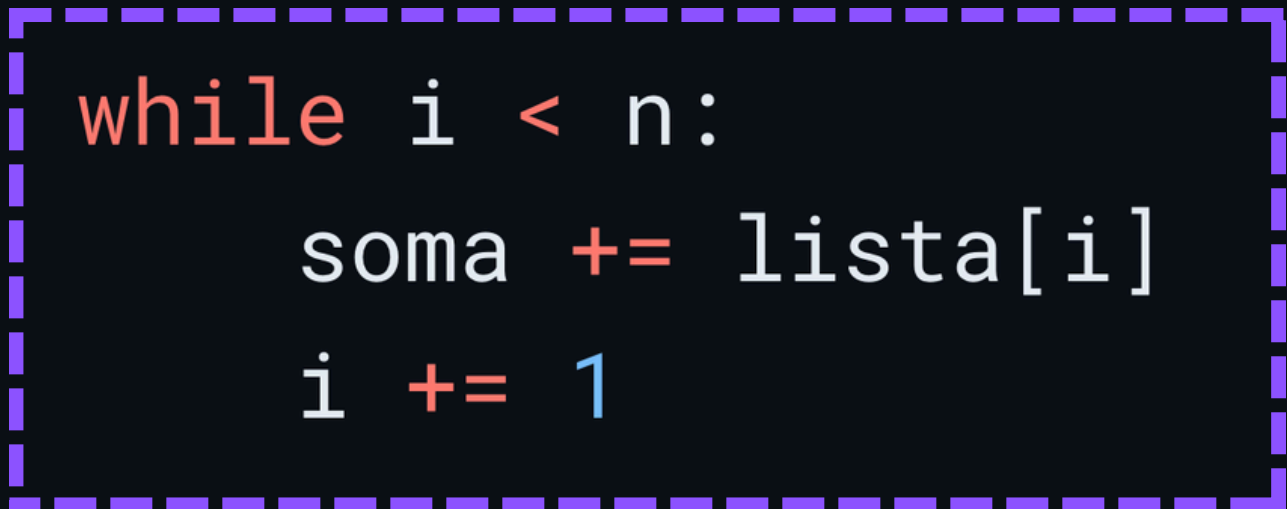
$$t(n) = 4 + 3n$$

n	t(n)
10	34
20	64
30	94



**temos uma subida linear. conforme
sobe n, sobe t(n) de forma linear.**


```
def soma_elementos(lista, n):  
    soma = 0  
    i = 0  
  
    while i < n:  
        soma += lista[i]  
        i += 1  
  
    return soma
```



A maior parte dos passos executados vem sempre do **laço de repetição**, pois é onde se concentra a **maior complexidade** de um algoritmo.

Notação assintótica

- Para a notação assintótica, a presença de constantes não afetam de forma significativa o resultado final.
- E isso acontece pois sempre consideraremos valores altos para n .
- Por exemplo, se tivéssemos um trilhão de passos. Ao final, teríamos três trilhões + 4. Será que esse 4 faz diferença?

$$t(n) = 4 + 3n$$

Logo... simplificamos.

$$t(n) = 4 + 3n$$

Removeremos todas as constantes (números que estão multiplicando ou somando)

$$t(n) = n$$

E pensando em Big O, diremos que:

O(n), que representa uma complexidade linear!

Passos gerais

1. Foque nas **repetições (loops)** do seu algoritmo.
2. Verifique a **complexidade dos métodos de terceiros** (se utilizados)
3. Foque no **termo de maior grau**, ignorando os termos de menor grau e as constantes.

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def exibe_elemento(lista, i):  
    print("O elemento lista[i] é: ")  
    print(lista[i])
```

1

1

podemos afirmar que

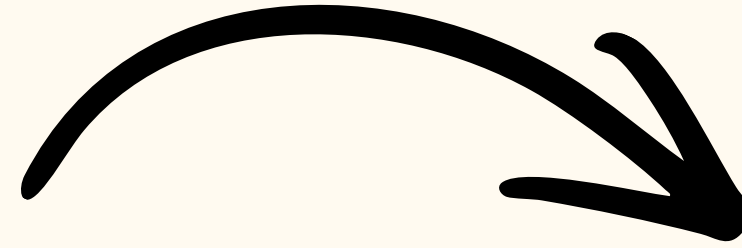
$$t(n) = 2$$

podemos afirmar que

$$t(n) = 2$$

logo, se tivermos uma
entrada de 10 elementos

$$t(10) = 2$$

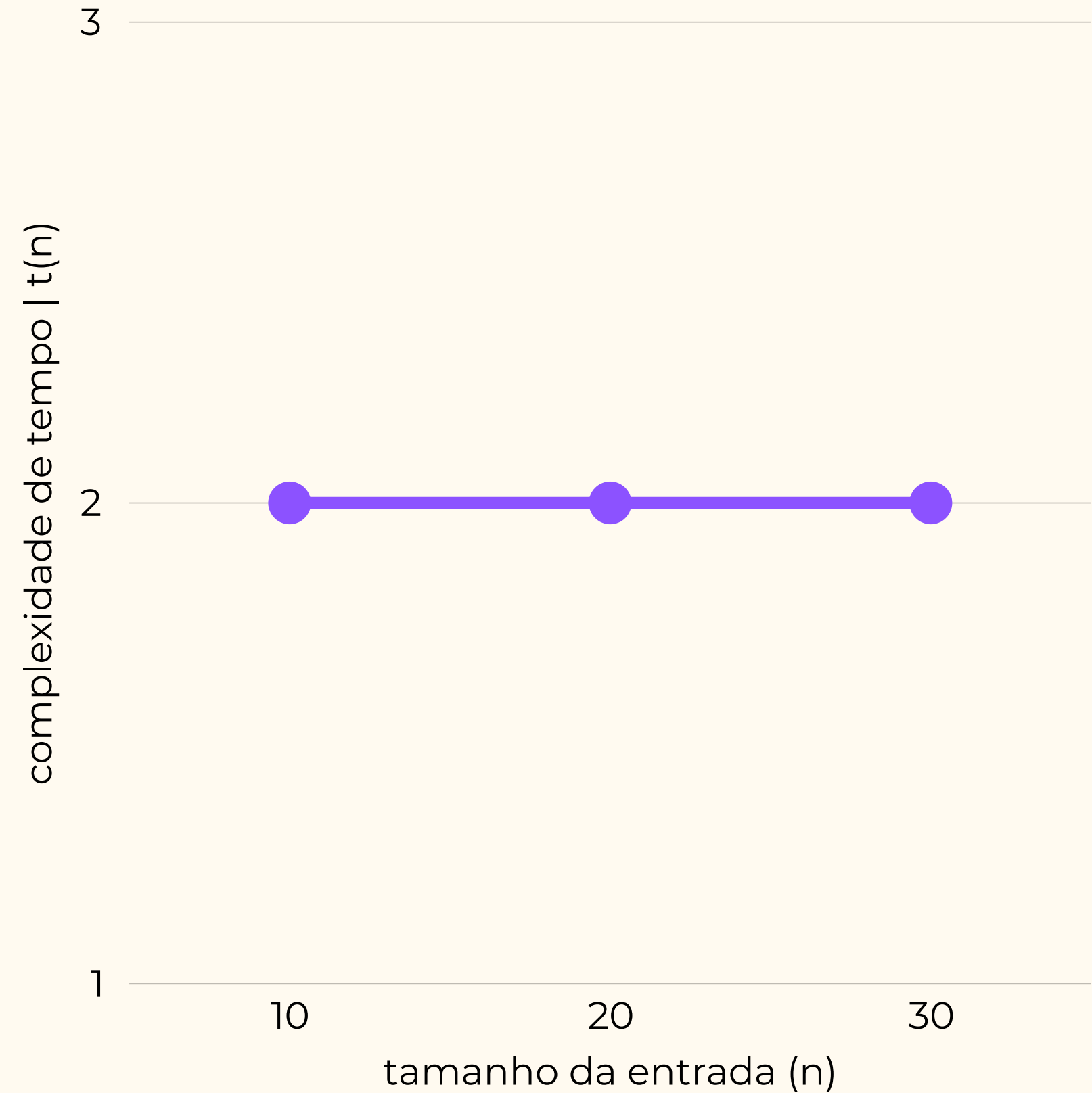


e assim por diante...

n	t(n)
10	2
20	2
1000	2

$$t(n) = 2$$

n	t(n)
10	2
20	2
30	2



temos uma constante. não importa o valor de n, a complexidade é a mesma!

Logo... simplificamos.

$$t(n) = 2$$

Removeremos todas as constantes... e se nada sobrar, diremos que é 1.

$$t(n) = 1$$

E pensando em Big O, diremos que:

O(1), que representa uma complexidade constante!

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def ache_elemento(array, n, elemento):  
    i = 0  
  
    while i < n:  
        if array[i] == elemento:  
            return True  
        i += 1  
  
    return False
```

Depende...

Na **melhor** das hipóteses:

$O(1)$

Na **pior** das hipóteses:

$O(n)$

Vale dizer que, em algumas literaturas, a
melhor possibilidade será retrata com
um omêga

Ω

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def ache_elemento(array, n, elemento):  
    i = 0  
  
    while i < n:  
        if array[i] == elemento:  
            return True  
        i += 1  
  
    return False
```

Por exemplo:

3	4	8	6	0
---	---	---	---	---

Na **melhor** das hipóteses, a busca será pelo elemento 3. Como é o primeiro elemento, o **algoritmo dará um 1 passo** no while.

Na **pior** das hipóteses, a busca será por um **elemento que não existe** (exemplo: 7) e o while será executado n vezes.

Qual a **complexidade do pior caso** desse algoritmo? Ou seja, quantos passos ele dá?

```
def ache_elemento_mat(mat, m, n, elem):  
    i = 0  
  
    while i < m:  
        j = 0  
  
        while j < n:  
            if mat[i][j] == elem:  
                return True  
            j += 1  
  
        i += 1  
  
    return False
```



m

n

podemos afirmar que

$$t(n) = m \times n$$

ou, se considerarmos
que falamos **n linhas** e
n colunas...

$$t(n) = n \times n$$

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def ache_elemento_mat(mat, m, n, elem):  
    i = 0  
  
    while i < m:  
        j = 0  
  
        while j < n:  
            if mat[i][j] == elem:  
                return True  
            j += 1  
  
        i += 1  
  
    return False
```



m

n

Na **melhor** das hipoteses:

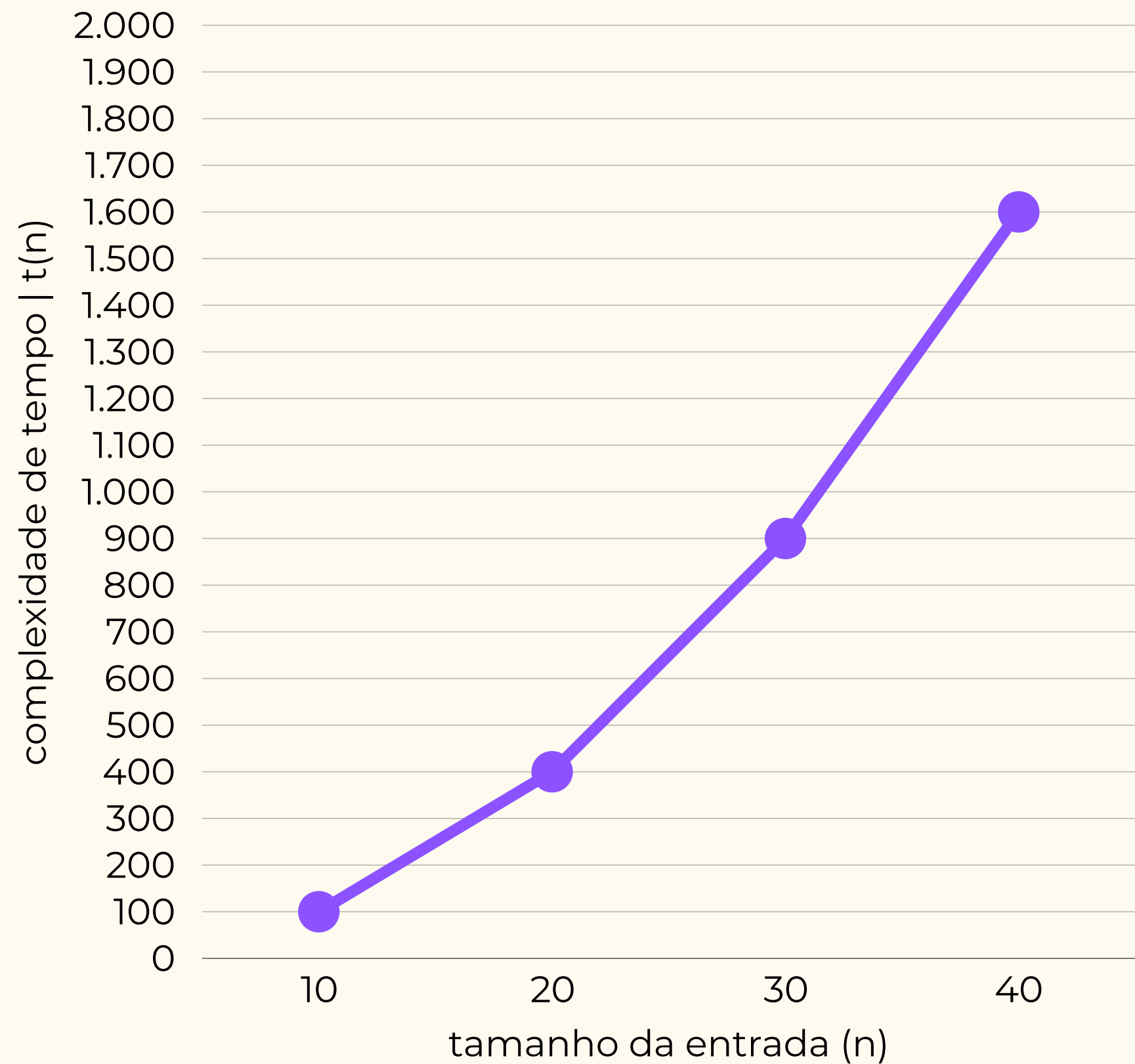
$O(1)$

Na **pior** das hipoteses:

$O(n^2)$

$$t(n) = n^2$$

n	t(n)
10	100
20	400
30	900

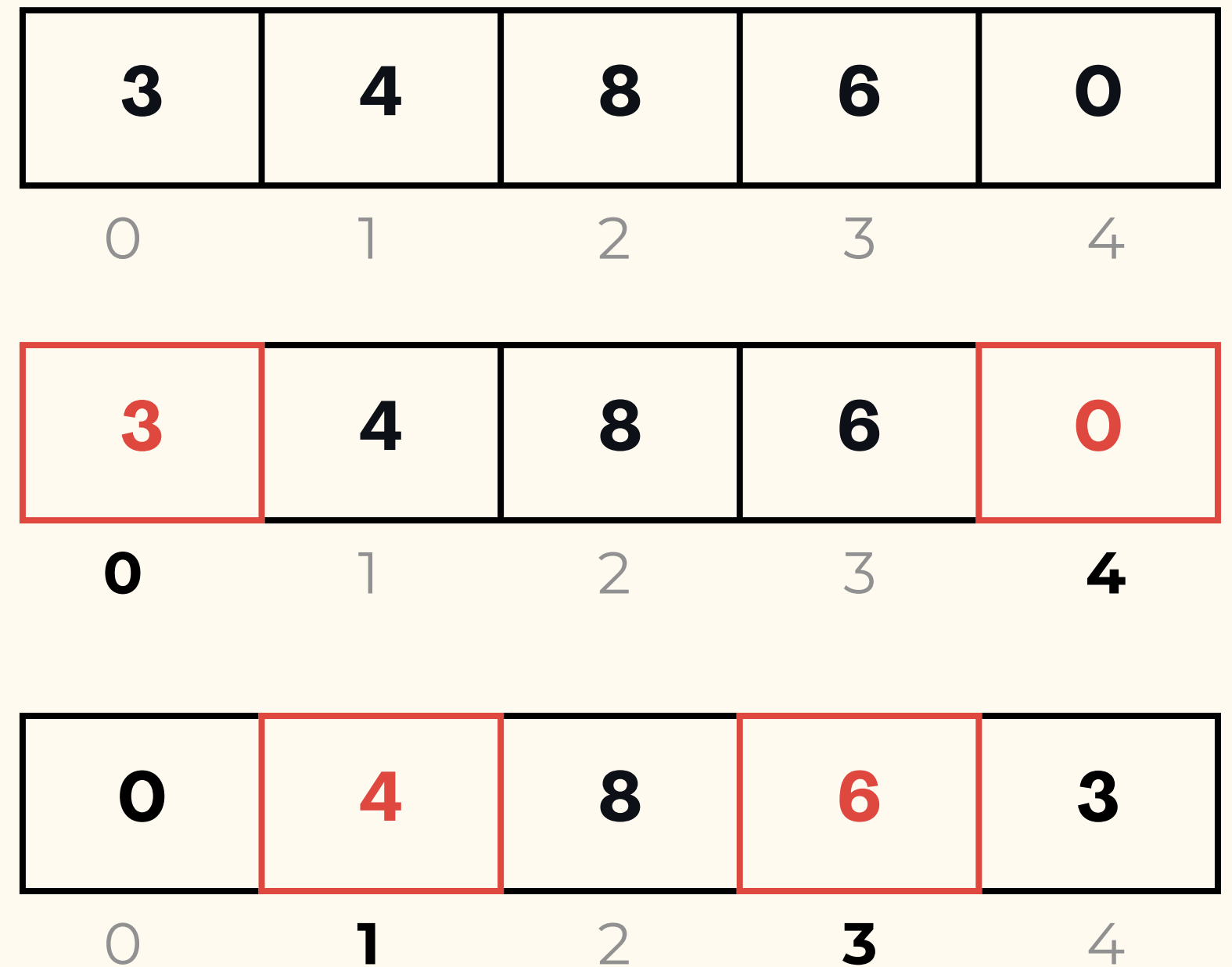


temos uma **complexidade quadrática**,
onde o **tempo evolui significativamente**
de acordo com a entrada

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def inverte(array, n):  
    start = 0  
    end = n - 1  
  
    while start < end:  
        aux = array[start]  
        array[start] = array[end]  
        array[end] = aux  
  
        start += 1  
        end -= 1  
  
    return array
```

Por exemplo:



Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def inverte(array, n):  
    start = 0  
    end = n - 1  
  
    while start < end:  
        aux = array[start]  
        array[start] = array[end]  
        array[end] = aux  
  
        start += 1  
        end -= 1  
  
    return array
```

Por exemplo:

0	6	8	4	3
0	1	2	3	4

0	6	8	4	3
0	1	2	3	4

Esse algoritmo será executado a metade de n . Ou seja, $n/2$.

podemos afirmar que

$$t(n) = \mathbf{n/2}$$

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def inverte(array, n):  
    start = 0  
    end = n - 1  
  
    while start < end:  
        aux = array[start]  
        array[start] = array[end]  
        array[end] = aux  
  
        start += 1  
        end -= 1  
  
    return array
```

Simplificando...

podemos afirmar que

$$t(n) = n/2$$

removendo as constantes

$$t(n) = n$$

ou seja

$$O(n)$$

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def media_mediana(array):  
    n = len(array)  
  
    media = 0.0  
    i = 0  
    while i < n:  
        media += array[i]  
        i += 1  
    media /= n  
  
    array_ord = sorted(array)  
  
    if n % 2 == 0:  
        med1 = array_ord[n // 2]  
        med2 = array_ord[n // 2 - 1]  
        mediana = (med1 + med2) / 2  
    else:  
        mediana = array_ord[n // 2]  
  
    return media, mediana
```

Neste caso, **temos dois blocos de código, com estruturas diferentes**. Vamos analisar por partes.

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def media_mediana(array):  
    n = len(array)  
  
    media = 0.0  
    i = 0  
    while i < n:  
        media += array[i]  
        i += 1  
    media /= n  
  
    array_ord = sorted(array)  
  
    if n % 2 == 0:  
        med1 = array_ord[n // 2]  
        med2 = array_ord[n // 2 - 1]  
        mediana = (med1 + med2) / 2  
    else:  
        mediana = array_ord[n // 2]  
  
    return media, mediana
```



n

Nesta primeira parte, **o algoritmo calcula a média**. Neste caso, ele vai percorrer todos os elementos da nossa lista. Temos uma complexidade **O(n)**.

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def media_mediana(array):  
    n = len(array)
```

```
    media = 0.0
```

```
    i = 0
```

```
    while i < n:
```

```
        media += array[i]
```

```
        i += 1
```

```
    media /= n
```

```
    array_ord = sorted(array)
```

```
    if n % 2 == 0:
```

```
        med1 = array_ord[n // 2]
```

```
        med2 = array_ord[n // 2 - 1]
```

```
        mediana = (med1 + med2) / 2
```

```
    else:
```

```
        mediana = array_ord[n // 2]
```

```
    return media, mediana
```

Aqui temos uma peculiaridade...
Lembra que um dos passos era verificar os métodos da linguagem? **Neste caso usamos o “sorted” e precisamos descobrir qual sua complexidade.**

Como? **Google it!**



$O(n \log n)$

podemos afirmar que

$$t(n) = n + n \log n$$

Qual a **complexidade** desse algoritmo?
Ou seja, quantos passos ele dá?

```
def media_mediana(array):  
    n = len(array)  
  
    media = 0.0  
    i = 0  
    while i < n:  
        media += array[i]  
        i += 1  
    media /= n  
  
    array_ord = sorted(array)  
  
    if n % 2 == 0:  
        med1 = array_ord[n // 2]  
        med2 = array_ord[n // 2 - 1]  
        mediana = (med1 + med2) / 2  
    else:  
        mediana = array_ord[n // 2]  
  
    return media, mediana
```

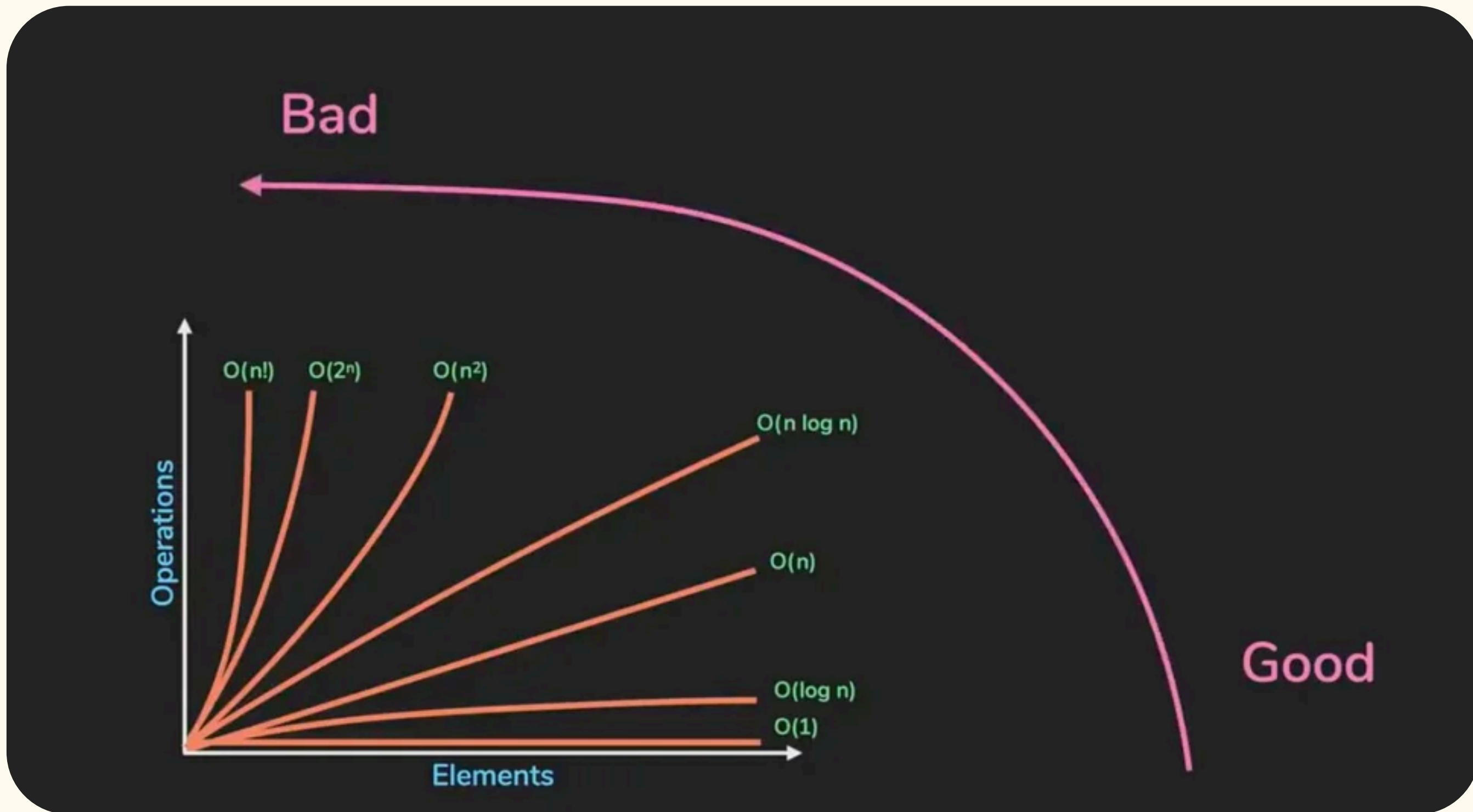
$$t(n) = n + n \log n$$

Aplicando a **terceira regra**, que diz que devemos ficar com o mais complexo, **ignoramos $O(n)$** , logo

podemos afirmar que

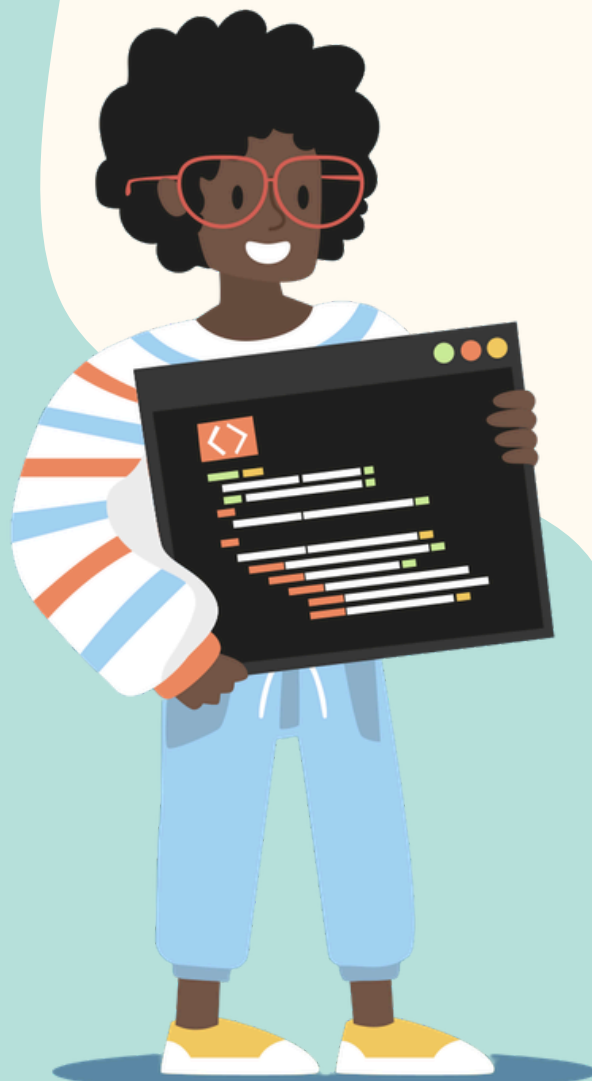
$$t(n) = n \log n$$

Ordem de complexidade



Prática 1

Dado o algoritmo ao lado,
defina a sua/suas
complexidade/s:



```
def tem_duplicacao(array, n):  
    i = 0  
  
    while i < n:  
        val = array[i]  
        j = i + 1  
  
        while j < n:  
            if array[j] == val:  
                return True  
            j += 1  
  
        i += 1  
  
    return False
```


Prática 2

Dado o algoritmo ao lado,
defina a sua/suas
complexidade/s:



```
def ache_min(array, n):  
    minimo = array[0]  
    i = 0  
  
    while i < n:  
        if array[i] < minimo:  
            minimo = array[i]  
        i += 1  
  
    return minimo
```

Prática 3

Dado o algoritmo ao lado,
defina a sua/suas
complexidade/s:



```
def remover(lista, i):  
    lista.pop(i)
```


Obrigado!



Quer trocar mais uma ideia? Me chama!

vhlopes@senacrs.com.br